EE4032 – Blockchain Engineering

Assignment: Zero Knowledge Proofs and Consensus Mechanisms

Name: Arnav Salkade Student Number: a0273649n

Exercise 5.1 – Zero Knowledge Proof

We work inside a large *prime-order* subgroup $G = \langle g \rangle$ of $(\mathbb{Z}/p\mathbb{Z})^*$, with prime order $q \mid (p-1)$. All exponents are taken modulo q. The prover's secret is $w \in \mathbb{Z}_q$ and the public value is $X = g^w \bmod p$. Intuitively, w is the private key and X plays the role of a public key.

(a) Why g^w reveals nothing about w

Publishing $X = g^w \bmod p$ does not reveal w because recovering w from (g, X) is the **discrete logarithm problem** in G. Exponentiation is easy, but inversion is believed intractable at standard sizes. With parameters such as a 3072-bit p and a 256-bit prime q, the best known attacks remain computationally infeasible. In short: many secrets could map to outputs that look just as random, and there is no known efficient way to go backwards.

(b) Protocol to prove knowledge of w (Schnorr identification)

The idea is simple: the prover commits to a random throwaway value, the verifier asks for a fresh challenge, and the prover answers in a way that only someone who knows w can. Formally:

$$\text{Prover picks } r \leftarrow \mathbb{Z}_q,\ a = g^r \ (\text{mod}p) \ \text{ and sends } a.$$

$$\text{Verifier sends a random challenge } c \leftarrow \mathbb{Z}_q.$$

$$\text{Prover replies } z = r + c\,w \ (\text{mod}q).$$

The verifier checks $g^z \stackrel{?}{=} a \cdot X^c \ (\text{mod}p)$. Completeness is immediate: $g^z = g^{r+cw} = g^r(g^w)^c = aX^c$, so an honest prover always passes.

(c) Why the verifier learns no information about w

This protocol is **honest-verifier zero knowledge**. What the verifier sees are (a, c, z), and those could have been generated without w: pick $c, z \leftarrow \mathbb{Z}_q$ uniformly and set $a = g^z X^{-c}$. This simulated transcript satisfies the same verification equation and has the same distribution as a real one. Therefore, from the verifier's point of view, no information about w is leaked beyond the fact that the prover is able to answer correctly.

(d) Why a dishonest prover cannot convince the verifier

Soundness rests on the fact that the challenge is unpredictable at commit time. If a cheating prover ever produced two accepting responses for the same commitment a but different challenges $c \neq c'$, namely $z = r + cw$ and $z' = r + c'w$, then

$$g^{z-z'} = X^{\,c-c'} = g^{w(c-c')} \;\Rightarrow\; w \equiv (z - z')(c - c')^{-1} \pmod{q}.$$

So two valid answers reveal w. Since the verifier chooses c after seeing a, a prover who does not know w cannot reliably produce correct answers to fresh random challenges.

(e) Fiat–Shamir: non-interactive version

To remove interaction, the random challenge is derived by hashing the public values. Let H be a cryptographic hash. The prover computes

$$a = g^r, \qquad c = \mathrm{I2B}_{\mathbb{Z}_q}\big(H(\mathsf{ZK} \parallel p \parallel g \parallel X \parallel a)\big), \qquad z = r + cw \pmod{q},$$

and publishes the proof (a, z). Anyone can recompute c from the same inputs and check $g^z = aX^c$. The hash output is deterministic (so everyone gets the same c) yet unpredictable before a is fixed. Security is argued in the **Random Oracle Model** (via the forking lemma): if an adversary could forge such proofs, one could obtain two different challenges for the same a and extract w as above.

Exercise 5.2 – Proof of Stake

(a) Why the condition $h(x) < T$ is not a valid PoW target

Searching for any x with $h(x) < T$ is not tied to the current block instance. Miners could precompute a warehouse of lucky x values and reuse them later. Binding the input to the block header, for example $h(B \parallel M \parallel x)$ where B is the previous block hash and M is the Merkle root, forces miners to expend fresh work per block.

(b) Why two blocks cannot both be validated if the adversary controls $< 1/3$

A block is valid if at least two-thirds of validators sign it. Two sets each larger than $2/3$ of the total must overlap in more than $1/3$ of validators. If fewer than $1/3$ are corrupt, the overlap includes an honest validator, and honest validators never sign two conflicting blocks in the same round. Thus two different blocks cannot both gather $2/3$ signatures in one round (standard quorum-intersection safety).

(c) Why $h(B \parallel M \parallel pk_i \parallel r) < T$ favors compute and risks corruption

If eligibility is defined by such an inequality, a wealthy adversary can try many public keys pk_i or many tweakable inputs per round to increase their chance of success; more hardware means more trials and a higher win rate. Worse, because the condition is publicly checkable, prospective winners can be predicted and targeted for bribery or censorship. To avoid this, eligibility should be derived from a unique, stake-bound key and an unpredictable per-round value.

(d) Why VRF properties matter in PoS

A verifiable random function provides three guarantees that align exactly with PoS needs. *Verifiability* lets anyone check a leader's claim without trust. *Uniqueness* ensures a validator has only one valid outcome per round, removing the incentive to grind multiple outputs for the same stake. *Unpredictability* prevents others from knowing the outcome in advance, blocking pre-targeting, bribery, and denial of service. With eligibility defined as $\text{Eval}(sk, r) < T \cdot \text{stake}$, selection becomes stake-proportional, checkable, and resistant to compute advantage.

(e) Why the signature-based construction is a VRF

Define

$$\text{Eval}(sk, x) = (y, \pi) = (H(\pi),\ \pi) \quad \text{where} \quad \pi = \text{Sign}(sk, x),$$

and verify with

$$\text{Ver}(pk, x, y, \pi) = \mathbf{1}\{\text{Verify}(pk, x, \pi) = 1 \ \wedge \ y = H(\pi)\}.$$

It is *verifiable* because anyone can check π and recompute y. It has *uniqueness* because a unique-signature scheme yields at most one valid π for a given (pk, x), hence at most one y. It is *unpredictable* to parties without sk because forging π would break signature unforgeability, and $H(\pi)$ is pseudorandom until π is known. This meets the VRF definition (in the random-oracle model for H).

Exercise 5.3 – DAO for On-Chain Randomness

A blockchain is deterministic, so a smart contract cannot pull randomness from thin air. A practical approach is a commit–reveal scheme with staking, penalties, and an optional verifiable delay to remove timing bias.

Rules and flow (per round r). Participants stake to join a committee. Each one draws a private random $s_{r,i}$ and posts a commitment $C_{r,i} = H(r \parallel s_{r,i})$ before the

commit deadline. After the commit phase ends, they reveal $s_{r,i}$. The contract checks $H(r \parallel s_{r,i}) = C_{r,i}$. All valid reveals are combined as

$$S_r \;=\; \bigoplus_i s_{r,i}, \qquad R_r \;=\; H(r \parallel S_r).$$

If at least one reveal is honest and unknown beforehand, the XOR remains uniformly random from the adversary's view; hashing it yields a public, unbiased R_r. Everyone can recompute the same value, so transparency is built in.

Incentives, security, and transparency. To discourage manipulation, anyone who commits but fails to reveal is slashed and forfeits the round reward; honest revealers are rewarded. Using r inside the commitment prevents replay across rounds. To mitigate the "last revealer" withholding attack, the contract can either apply a verifiable delay function to S_r before finalizing R_r (so outcomes cannot be tested instantly), or strongly slash non-revealers while allowing third parties to relay a participant's already-disclosed secret. All events (commits, reveals, penalties, outputs) are on-chain, so the process is auditable end to end.

Why this works. XOR of at least one uniform, independent input is uniform. Therefore if any single participant behaves honestly in a round, the combined seed is unpredictable to adversaries. Hashing the aggregate fixes length, hides structure, and makes the output easy to use as a seed for other protocols.